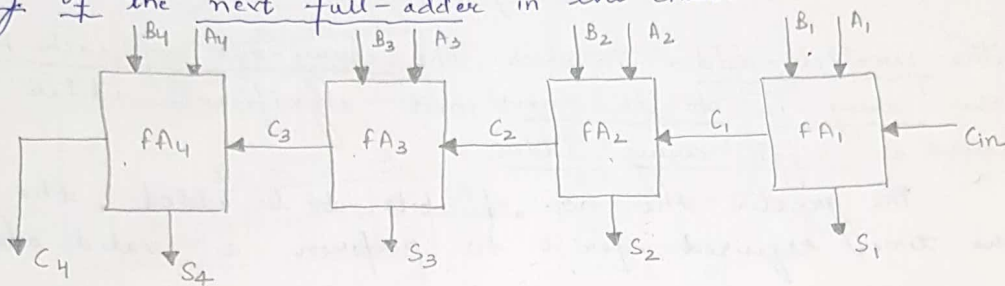


Binary Parallel Adder -

A binary parallel adder is a digital circuit that adds two binary numbers in parallel form & produces the arithmetic sum of those numbers in parallel form.

It consists of full adders connected in a chain, with the o/p carry from each full-adder connected to the i/p carry of the next full-adder in the chain.



Logic diagram of a 4-bit binary parallel adder

∴ all the augend & addend bits are fed into the circuit simultaneously & additions in each position are taking place at the same time, this circuit is known as a parallel adder.

An n-bit parallel adder requires n full adders & it can be constructed from 4-bit, 2-bit, and 1-bit FA ICs by cascading several packages.

The Ripple Carry Adder -

As discussed above, carry-out of each stage is connected to carry-in of the next stage. The sum & carry-out bits of any stage cannot be produced, until some time after the carry-in of that stage occurs i.e. due to propagation delay of the previous full-adder.

for ex:- S_1 & C_1 are not valid, given by FA₁, until after the propagation delay of FA₁.

S_2 & C_2 given by FA₂ are not valid after the cumulative propagation delay of 2 FAs (FA₁ & FA₂).

At each stage, sum bit is not valid until after the carry bit in all the preceding stages are valid. The carry bits must propagate or ripple through all stages before the most significant sum bit is valid.

Thus, total sum (or parallel output) is not valid until after the cumulative delay of all the adders.

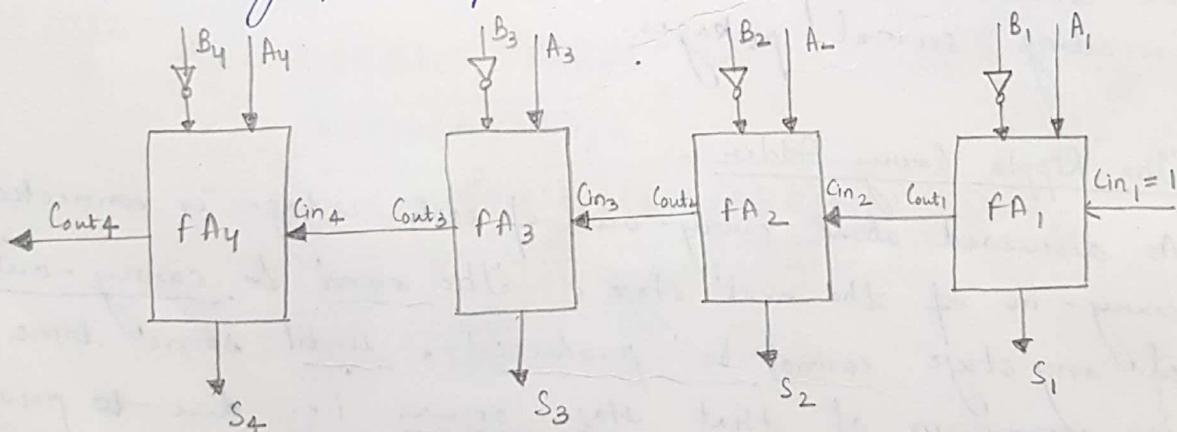
The parallel-adder in which the carry-out of each FA is the carry-in to the next most significant adder is called a ripple carry adder.

The greater the no. of bits to be added, the greater the time required for it to perform a valid addition.

4-bit parallel subtractor $\Rightarrow$

$$\begin{array}{r} A \text{ (minuend)} \\ - B \text{ (subtrahend)} \\ \hline \end{array} \Rightarrow + \frac{A}{\text{2's complement of } B} \Rightarrow \left\{ \begin{array}{l} \text{1's complement} \\ + 1 \\ \hline \text{2's complement} \end{array} \right.$$

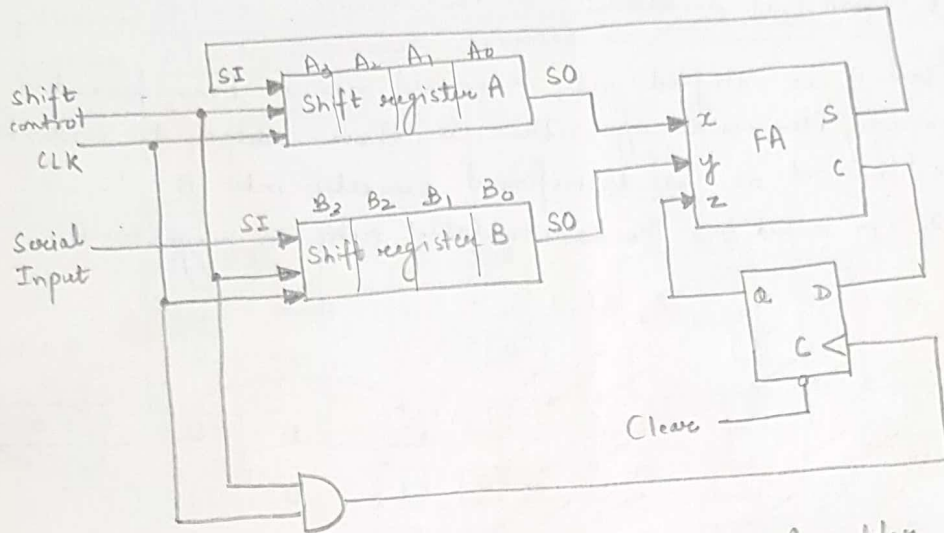
We will use the fact that, $A-B$ can be done by taking 2's complement of B & adding it to A . Also, 2's complement can be obtained by taking 1's complement & adding 1 to least significant pair of bits.



Logic diagram of a 4-bit parallel subtractor

Serial adder - (only one pair of bits is added at a time)

1. A serial adder is used to add binary numbers in serial form.
2. Two binary nos. to be added serially are stored in two shift registers - A & B.
3. Bits are added one pair at a time through a single FA circuit.



Logic diagram of a serial adder

4. Out of FA is transferred to D F-F. Op of this FF is then used as a carry i/p for next pair of significant bits.
 5. Sum of FA can be transferred to a 3rd shift register. But it is possible to use one register for storing both augend & sum bits. Shift the sum into A while bits of A are shifted out.
- Also, B can be used to transfer a new binary no. when addend bits are shifted out.

Operation of Serial adder -

Initially, A holds - augend
B holds - addend

$$DFF (\text{carry FF}) = 0$$

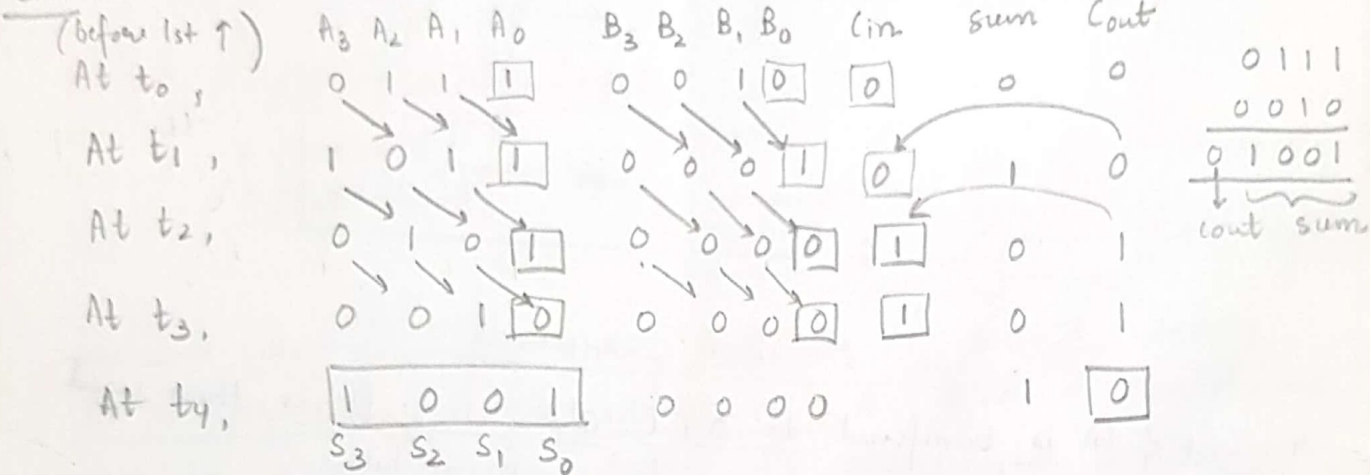
o/ps (SO) of A & B provide pair of bits for FA at x & y. Shift control enables both registers & carry FF. $\therefore$, at the clock pulse both registers are shifted once to the right, sum

bit from S enters left most FF of A & carry is transferred into FF.

Shift control enables registers for a no. of clock pulses = no. of bits of register. For each succeeding clock pulse, new sum bit is transferred to A, new carry to C & both registers are shifted to right. This process continues until the shift control is disabled.

Also, while B is shifted out, a second no. is transferred to it through its serial i/p. This is then added to contents of A while a 3rd no. is transferred serially into B. $\therefore$, we can add 2, 3 or more nos. & accumulate sum in register A.

ex: -



Difference b/w Serial & Parallel Adders -

<u>Parallel Adder</u>	<u>Serial Adder</u>
1. It uses register with parallel load. / bits are added simultaneously.	1. It uses shift registers. (Add'n is performed bit by bit starting from LSB)
2. No. of FA circuits = no. of bits in the binary nos. (more components are required)	2. It requires only one FA & a D FF (or carry FF) (fewer components are required)
3. It is a combinational circuit.	3. It is a sequential circuit.
4. It produces sum & cout in one clock pulse (with some propagation delay). $\Rightarrow$ <u>Faster</u>	4. It produces sum & cout in clock pulses = no. of bits. (ex:- In 4 clock pulses for 4-bit nos.) $\Rightarrow$ <u>Less fast</u>

✓ The look-ahead-carry adder speeds up the process by eliminating this ripple carry delay. It examines all the i/p bits simultaneously & also generates the carry-in bits for all the stages simultaneously.

This method of speeding up addition process is based on two additional functions of FA -

- i) the carry generate function
- ii) the carry propagate functions.

We know that,

full adder - is made up of two half adders
↳ Half adder - contains X-OR gate for sum [$\text{sum} = A \oplus B$]
AND gate for carry [$\text{Carry} = A \cdot B$]

Ref. to dig.

Case 1 :-

If both bits A_n & $B_n = 1$, a carry has to be generated in this stage regardless of whether i/p carry $C_n = 0$ or a 1. This is called generated carry.

This is expressed as:- $G_n = A_n \cdot B_n$

This carry appear at the o/p through the OR gate.

Case 2 :-

X-OR gate inside the HA at the i/p produces inter-mediate sum bit - called P_n (propagated carry).

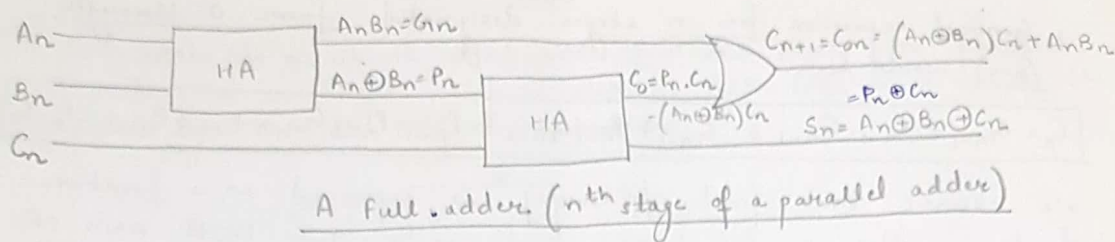
This is expressed as $P_n = A_n \oplus B_n$

Later explained why...

Next, P_n & C_n are added using X-OR gate inside the second half adder to produce the final sum & carry:-

$$S_n = P_n \oplus C_n = A_n \oplus B_n \oplus C_n$$

$$\hookrightarrow C_0 = P_n \cdot C_n = (A_n \oplus B_n) C_n \Rightarrow \text{becomes i/p carry for } (n+1)\text{th stage.}$$



Assume $P_n = 1$ and $C_n = 1 \Rightarrow$

If carry C_n is propagated to the o/p only if $P_n = 1$

$$\left\{ \begin{array}{l} \text{as, } S_n = P_n \oplus C_n = 1 \oplus 1 = 0 \\ C_0 = P_n \cdot C_n = 1 \cdot 1 = 1 \end{array} \right.$$

If $P_n = 0$ & even if $C_n = 1 \Rightarrow$

AND gate in 2nd HA will inhibit C_n i.e. $C_0 = P_n \cdot C_n = 0 \cdot 1 = 0$

$\therefore$, P_n is called propagated carry as it enables propagation of C_n to the carry o/p of the n^{th} stage, denoted by C_{n+1} or C_{out} .

$\therefore$, We can say that C_{n+1} (carry out of n^{th} stage) = 1 when either -

- i) $G_n = 1$ or $P_n \cdot C_n = 1$
- or ii) both G_n & $P_n \cdot C_n = 1$

final sum & carry o/p's of the n^{th} stage are given as :-

$$S_n = P_n \oplus C_n \quad \text{where } P_n = A_n \oplus B_n$$

$$C_{out} = C_{n+1} = G_n + P_n C_n \quad \text{where } G_n = A_n B_n$$

Observe the recursive nature of carry out of n^{th} stage (C_{out}). It is possible to express the o/p carry of a higher significant stage in terms of applied i/p variables A, B & carry-in to the LSB adder.

$\therefore$, we have,

$$\begin{aligned} C_1 &= G_0 + P_0 C_0 \\ C_2 &= G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0 \\ C_3 &= G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\ C_4 &= G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \\ C_5 &= G_4 + P_4 C_4 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 G_0 + P_4 P_3 P_2 P_1 P_0 C_0 \end{aligned}$$

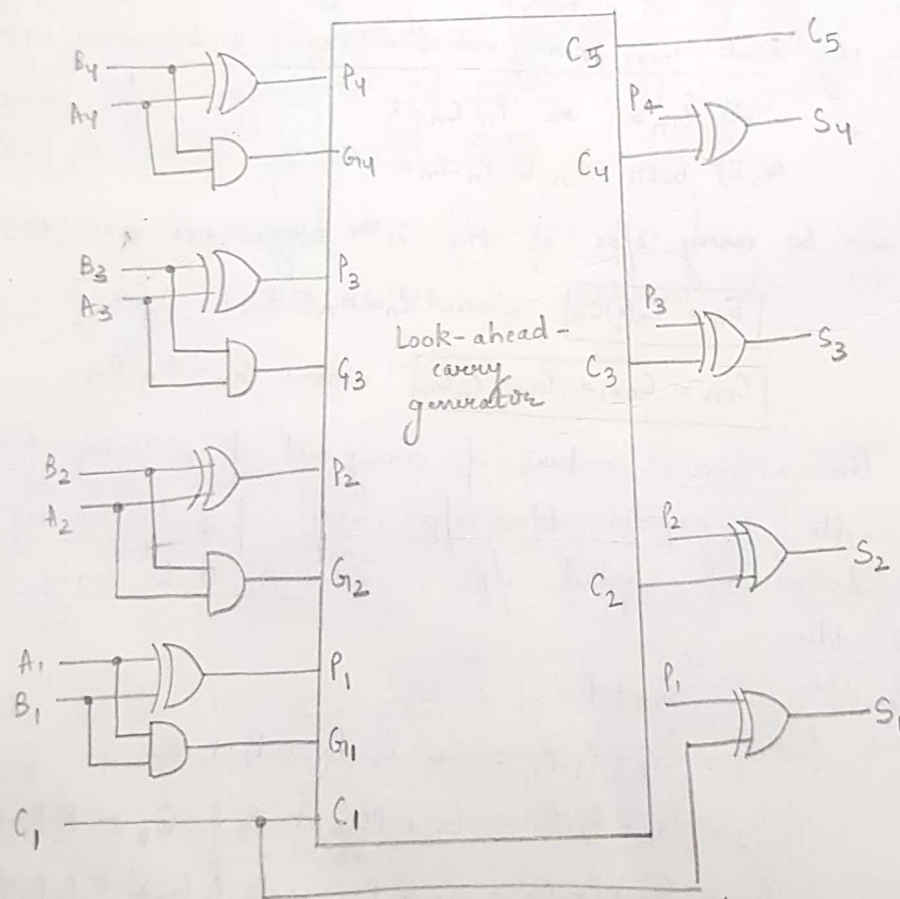
General expression for n stages designated from 0 through $(n-1)$ would be -

$$C_n = G_{n-1} + P_{n-1} \cdot G_{n-1} = G_{n-1} + P_{n-1} \cdot G_{n-2} + P_{n-1} \cdot P_{n-2} \cdot G_{n-3} + \dots + P_{n-1} \cdot \dots \cdot P_0 \cdot C_0$$

We observe that final o/p carry is expressed as a function of i/p variables in SOP form which is a two-level AND-OR or equivalent NAND-NAND form.

Imp. ∴, to produce o/p carry for any particular stage, it requires only that much time required for signals to pass through two levels only. Hence, circuit for look-ahead-carry introduces a delay corresponding to two gate levels.

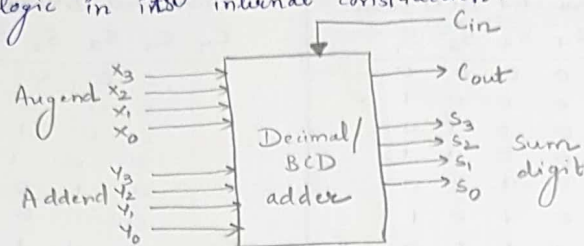
The block diagram of a 4-stage look-ahead-carry parallel adder is shown next :-



Logic diagram of a 4-bit look-ahead-carry adder

BCD Adder

A BCD adder is a circuit that adds two BCD digits in parallel and produces a sum digit which is also in BCD. It must include the correction logic in its internal construction.



Block diagram of a BCD adder

A BCD adder circuit must be able to do the following:-

- i) Add 4-bit BCD code groups using straight binary addition.
- ii) If 4-bit sum is equal to or less than 9, sum is in proper BCD form and no correction is needed.
- iii) If 4-bit sum is greater than 9 or if carry is generated from the sum, the sum is not in the BCD form. A correction of 0110 should be added to that sum, to produce the proper BCD result. This will produce a carry to be added to the next decimal position.

∴, a BCD adder must be able to do the following:-

- i) Add two 4-bit BCD code groups (using straight binary addition).
- ii) Determine if $\text{sum} > 1001$, add 0110 (decimal 6) if it is to generate a carry to the next decimal position.

The first requirement is easily met by using 4-bit binary parallel adder (74LS83 IC). It adds two BCD code groups $A_4A_3A_2A_1$ and $B_4B_3B_2B_1$, the adder will output $C_4S_4S_3S_2S_1$.

The sum output $C_4S_4S_3S_2S_1$ can range from 00000 to 10011 (when both BCD groups are 1001 = 9 & $C_{in} = 1$). The circuitry for a BCD adder must include the logic to detect whenever

the sum is greater than 01001, so that correction can be added in.

All the cases are listed below :-

Decimal digit	Uncorrected BCD sum					Corrected BCD sum				
	C_4	S_4	S_3	S_2	S_1	C_4	S_4	S_3	S_2	S_1
0			0	0	0	0	0	0	0	0
1			0	0	0	0	0	0	0	1
2			0	0	1	0	0	0	1	0
3			0	0	1	0	0	0	1	1
4			0	1	0	0	0	1	0	0
5			0	1	0	0	0	1	0	1
6			0	1	1	0	0	1	1	0
7			0	1	1	0	0	1	1	1
8			1	0	0	1	0	0	0	0
9			1	0	0	1	0	0	0	1
10			1	0	1	1	0	0	0	0
11			1	0	1	1	0	0	0	1
12			1	1	0	1	0	0	1	0
13			1	1	0	1	0	0	1	1
14			1	1	1	1	0	1	0	0
15			1	1	1	1	0	1	0	1
16			0	0	0	1	0	1	1	0
17			0	0	0	1	0	1	1	1
18			0	0	1	1	1	0	0	0
19			0	0	1	1	1	0	0	1

No correction required

correction required

Let us define a logic o/p X that will go high when sum > 01001. If we examine these cases, we see that X will be HIGH for either of the following conditions :-

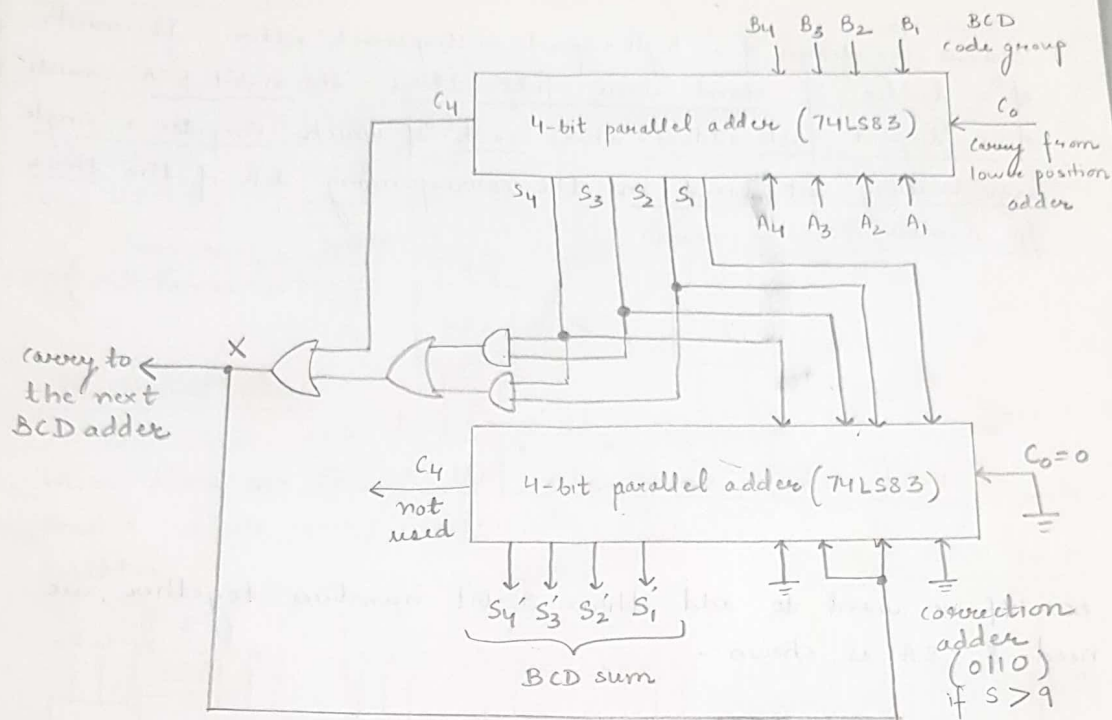
- Whenever $C_4 = 1$ (sum > 15)
- Whenever $S_4 = 1$ and either S_3 or S_2 or both are 1 (sums 10 to 15)

∴, this condition can be expressed as :-

$$X = C_4 + S_4 S_3 + S_4 S_2 = C_4 + S_4 (S_3 + S_2)$$

∴, when $X = 1$, correction factor (0110) must be added to the sum bits & it is necessary to generate a carry (right col.)

The complete circuitry for a BCD adder, including the logic circuit implementation for X is shown next :-

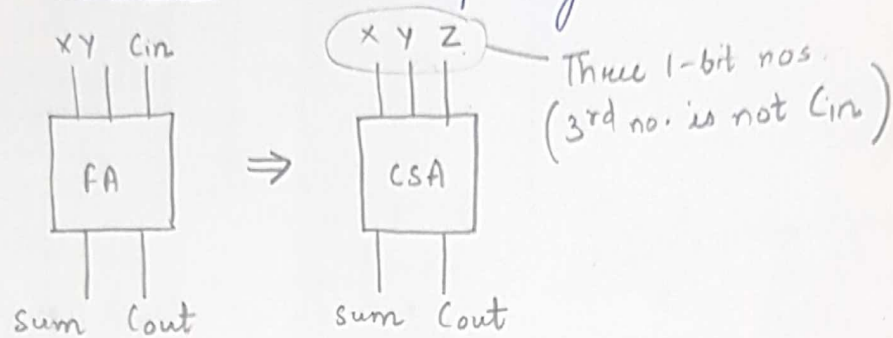


Logic diagram of a BCD adder using two 4-bit adders and a correction - detector circuit

- i) The two BCD code groups $A_4A_3A_2A_1$ and $B_4B_3B_2B_1$ are added together in the upper 4-bit adder, to produce the sum $C_4S_4S_3S_2S_1$.
- ii) The logic gates shown implement the expression for X.
- iii) The lower 4-bit adder will add the correction 0110 to the sum bits, only when $X=1$, producing the final BCD sum o/p $S_4'S_3'S_2'S_1'$.
- iv) X is also the carry-out that is produced when $\text{sum} > 01001$. When $X=0$, there is no carry & no addition of 0110.
- v) Two or more BCD adders can be connected in cascade when two or more digital nos. are to be added. The carry-out of the first BCD adder is connected as carry-in of the 2nd BCD adder and so on.

Carry Save Adder (CSA)

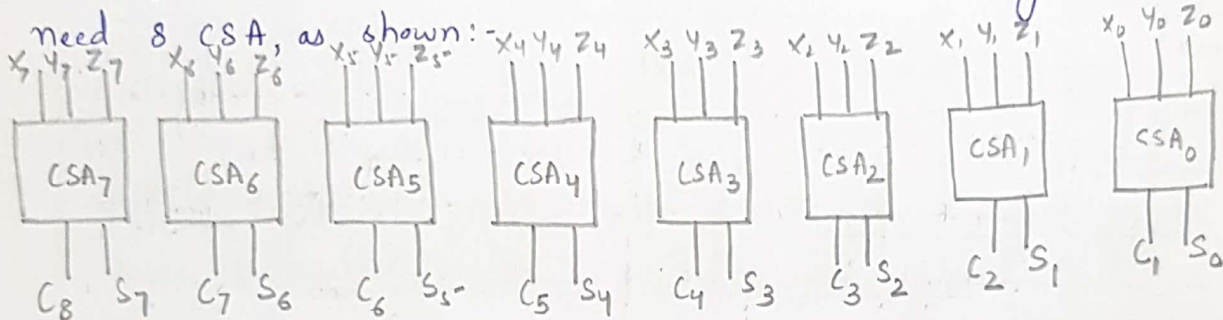
CSA is the design of a high-speed multioperand adder. It consists of a ladder of stand-alone full adders. The n -bit CSA consists of n -disjoint full adders where each of which computes a single sum & carry bit based on the corresponding bits of the three i/p numbers.



1-bit CSA is same as a full-adder

n -bit CSA -

ex:- If we want to add three 8-bit numbers together, we



Here, 8-bit CSA consists of 8 CSA or FAs. Unlike the normal adder such as ripple carry adder, a CSA consists of multiple (one-bit) without any carry chaining. Each C & S are computed independently & each S_i & C_{i+1} are computed independently from all other C & S .

$\therefore$ n -bit CSA consumes $3n$ -bit numbers to be added & produces two outputs - n -bit partial sum & n -bit carry as shown next :-

Calculation of CSA -

A	0 1 1 0 0	(12)	} = 43
+B	1 1 0 0 1	(25)	
+C	0 0 1 1 0	(6)	
sum bit	1 0 0 1 1	(19)	
carry bit	0 1 1 0 0	(24)	
final result	1 0 1 0 1 1	(43)	

A CSA may be implemented in several different ways. For binary operators,

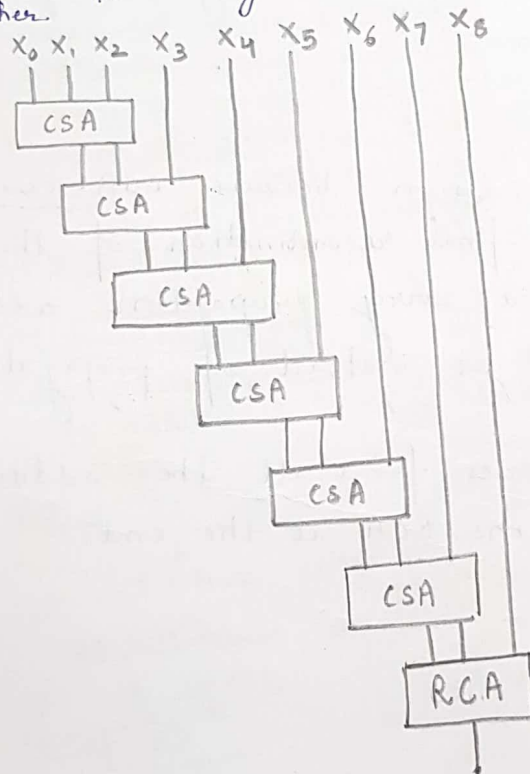
$$\text{sum} = (A+B+C) \bmod 2$$

$$\text{Cout} = \frac{(A+B+C) - \text{sum}}{2}$$

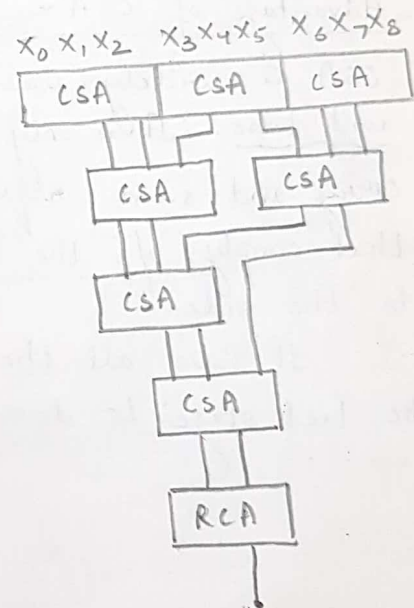
→ remainder of x/y

Multilevel CSA -

It uses a no. CSAs interconnected as a multilevel adder tree to add more than one no. per cycle. To get the final sum, we still need a ripple carry adder to add sum and carry vector together.



Multilevel Tree



Wallace Tree

To add k bits, $(k-2)$ CSA are required with each additional i/p operand increasing the no. of CSAs by one.

When adding together three or more numbers, using a carry-save adder followed by a ripple carry adder is faster than using two ripple carry adder. This is because a ripple carry adder cannot compute a sum bit without waiting for the previous carry bit to be produced & thus, has a delay equal to that of n full adders. However, a carry save adder produces all of its o/p values in parallel & thus has the same delay as a single full adder. Thus total computation time for a carry save adder plus a ripple carry adder = $\boxed{n+1}$, whereas for two ripple carry adder it would be $\boxed{2n}$.

Advantage of CSA -

CSA is widely used in design because CSA saves logic and time. It is only the final recombination of the final carry and sum require a carry propagation addition that simply o/p the carry bit instead of propagating them to the side.

It save all the carries from all the additions to the last stage & do the one RCA at the end.